

CS-112

Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

COURSE INSTRUCTOR : MS. HABIBA ARSHAD

Today we will talk about!

- Objects and Classes in Java
- Method in Java
- Parameters in Java

Introducing Classes and Objects

Classes and Objects in Java

In object-oriented programming technique, we design a program using objects and classes.

An object in Java is the physical as well as a logical entity, whereas, a class in Java is a logical entity only.

Classes in Java

Syntax:

```
access modifier class ClassName{  
    Field;  
    Method;  
}
```

Example



Class : Logical Entity



A class in Java is like a blueprint or design plan for a house.



The blueprint describes what the house should have (doors, windows, rooms) and what you can do with it (open doors, turn on lights). But you can't live in or touch a blueprint.



It's just a logical concept, a design.

```
class House {  
    int numOfRooms;  
    String color;  
  
    void openDoor() {  
        System.out.println("The  
door is opened.");  
    }  
}
```

Example



Object: Logical + Physical entity



An object is an actual house built from that blueprint.



You can see it, live in it, paint it, or open its door.



It's a real instance of that blueprint with specific values and behavior.

```
public class Main {  
    public static void main(String[] args) {  
        // Creating objects (real houses)  
        House myHouse = new House();  
        myHouse.numOfRooms = 3;  
        myHouse.color = "Blue";  
  
        House neighborHouse = new House();  
        neighborHouse.numOfRooms = 5;  
        neighborHouse.color = "Red";  
  
        // Interacting with the objects  
        System.out.println("My house has " + myHouse.numOfRooms + "  
rooms.");  
        myHouse.openDoor();  
  
        System.out.println("Neighbor's house is " +  
neighborHouse.color);  
        neighborHouse.openDoor();  
    }  
}
```

Classes and Objects

Object –

Objects have states/attributes and behaviors. Example: A dog has states - color, name, breed as well as behaviors – wagging the tail, barking, eating. An object is an instance of a class.

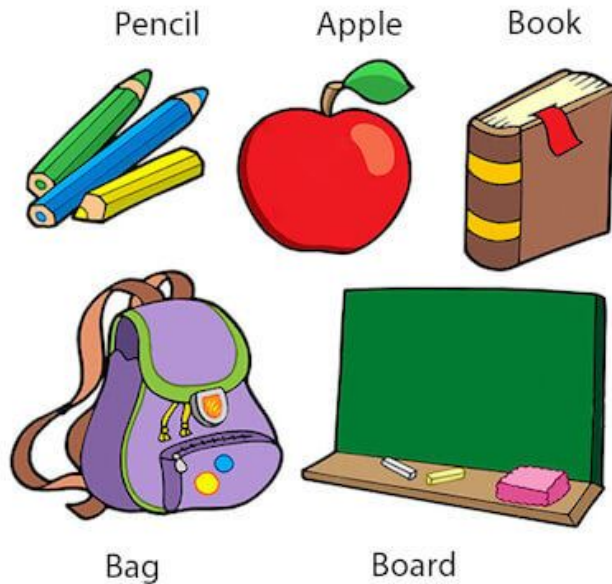
Class –

A collection of objects with same properties and functions is known as class.

A class can be defined as a template/blueprint that describes the behavior/state that the object of its type support.

Object in Java

Objects: Real World Examples



An entity that has state and behavior is known as an object e.g., chair, bike, marker, pen, table, car, etc.

It can be physical or logical (tangible and intangible).

The example of an intangible object is the banking system.

Characteristics of Object

- **State:** represents the data (value) of an object.
- **Behavior:** represents the behavior (functionality) of an object such as deposit, withdraw, etc.
- **Identity:** Identity means each object is unique, even if it has the same state and behavior as another object. An object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. However, This uniqueness is usually handled internally by the system (like the JVM) through something called an object identity.

Characteristics of Object

For Example,

Pen is an object.

Its name is Parker; color is white, known as its state.

It is used to write, so writing is its behavior.

Now, if you create two Pen objects with the same name and color, they still are two separate objects, each with its own identity (unique ID in memory). The JVM can tell them apart, even if to you they look the same.

Objects in Java

If we consider a dog, then its state is - name, breed, color, and the behavior is - barking, wagging the tail, running.

If you compare the software object with a real-world object, they have very similar characteristics.

Software objects also have a state and a behavior. A software object's state is stored in fields and behavior is shown via methods.

So, in software development, methods operate on the internal state of an object and the object-to-object communication is done via methods.

Real World Example

Consider a **Pen** as a real-world object.

- **State:** Brand = "Parker", Color = "White", Ink Level = 80%
- **Behavior:** Write, Refill
- **Identity:** Even if you have two identical pens, each is a distinct physical object.
- Software Object (Example Java Code)



```
public class Pen {  
    // State/ attributes (Fields)  
    String brand;  
    String color;  
    int inkLevel;  
  
    // Behavior/ Functionality (Methods)  
    public void write(String text) {  
        if (inkLevel > 0) {  
            System.out.println("Writing: " + text);  
            inkLevel -= 10; // reduces ink level each time you  
write  
        } else {  
            System.out.println("Cannot write, ink is empty.");  
        }  
    }  
    public void refill(int inkAmount) {  
        inkLevel += inkAmount;  
        System.out.println("Pen refilled. Current ink level: "  
+ inkLevel);  
    }  
}
```

```
// Main method to test Pen class  
public static void main(String[] args) {  
    Pen myPen = new Pen();  
    myPen.brand = "Parker";  
    myPen.color = "White";  
    myPen.inkLevel = 20;  
  
    System.out.println("Pen brand: " + myPen.brand);  
    System.out.println("Pen color: " + myPen.color);  
    System.out.println("Initial ink level: " +  
myPen.inkLevel);  
  
    myPen.write("Hello, world!");  
    System.out.println("Ink level after writing: " +  
myPen.inkLevel);  
  
    myPen.refill(30);  
    System.out.println("Ink level after refill: " +  
myPen.inkLevel);  
}  
}
```

Pen brand: Parker

Pen color: White

Initial ink level: 20

Writing: Hello, world!

Ink level after writing: 10

Pen refilled. Current ink level: 40

Ink level after refill: 40

Methods in Java

Class Methods

- A class can have any number of methods to access the value of various kinds of methods.
- In the given example, **barking()**, **hungry()** and **sleeping()** are methods.

```
public class Dog {  
    String breed;  
    int age;  
    String color;  
  
    void barking() {  
    }  
  
    void hungry() {  
    }  
  
    void sleeping() {  
    }  
}
```

Why Write Methods?

- Methods are commonly used to break down a problem into small manageable pieces. This is called *divide and conquer*.
- Methods simplify programs.
- If a specific task is performed in several places in the program, a method can be written once to perform that task, and then be executed anytime it is needed. This is known as *code reuse*.

Member Methods

In Java, a method is like a function which is used to expose the behavior of an object.

Advantage of Method

- Code Reusability
- Code Optimization

You can pass values to your methods

A method uses parameters

A caller passes arguments

If a method takes a parameter , you must pass it something of proper type

```
public class Dog
{
    public String name;
    public String breed;
    public int age;
    public void writeOutput()
    {
        System.out.println("Name: " + name);
        System.out.println("Breed: " + breed);
        System.out.println("Age in calendar years: " +
                           age);
        System.out.println("Age in human years: " +
                           getAgeInHumanYears());
        System.out.println();
    }
    public int getAgeInHumanYears()
    {
        int humanAge = 0;
        if (age <= 2)
        {
            humanAge = age * 11;
        }
        else
        {
            humanAge = 22 + ((age-2) * 5);
        }
        return humanAge;
    }
}
```

Later in this chapter we will see that the modifier **public** for instance variables should be replaced with **private**.

Example Code

```
public static void main(String[] args)
{
    Dog balto = new Dog();
    balto.name = "Balto";
    balto.age = 8;
    balto.breed = "Siberian Husky";
    balto.writeOutput();

    Dog scooby = new Dog();
    scooby.name = "Scooby";
    scooby.age = 42;
    scooby.breed = "Great Dane";
    System.out.println(scooby.name + " is a " +
                       scooby.breed + ".");
    System.out.print("He is " + scooby.age +
                    " years old, or ");

    int humanYears = scooby.getAgeInHumanYears();
    System.out.println(humanYears + " in human years.");
}
}
```

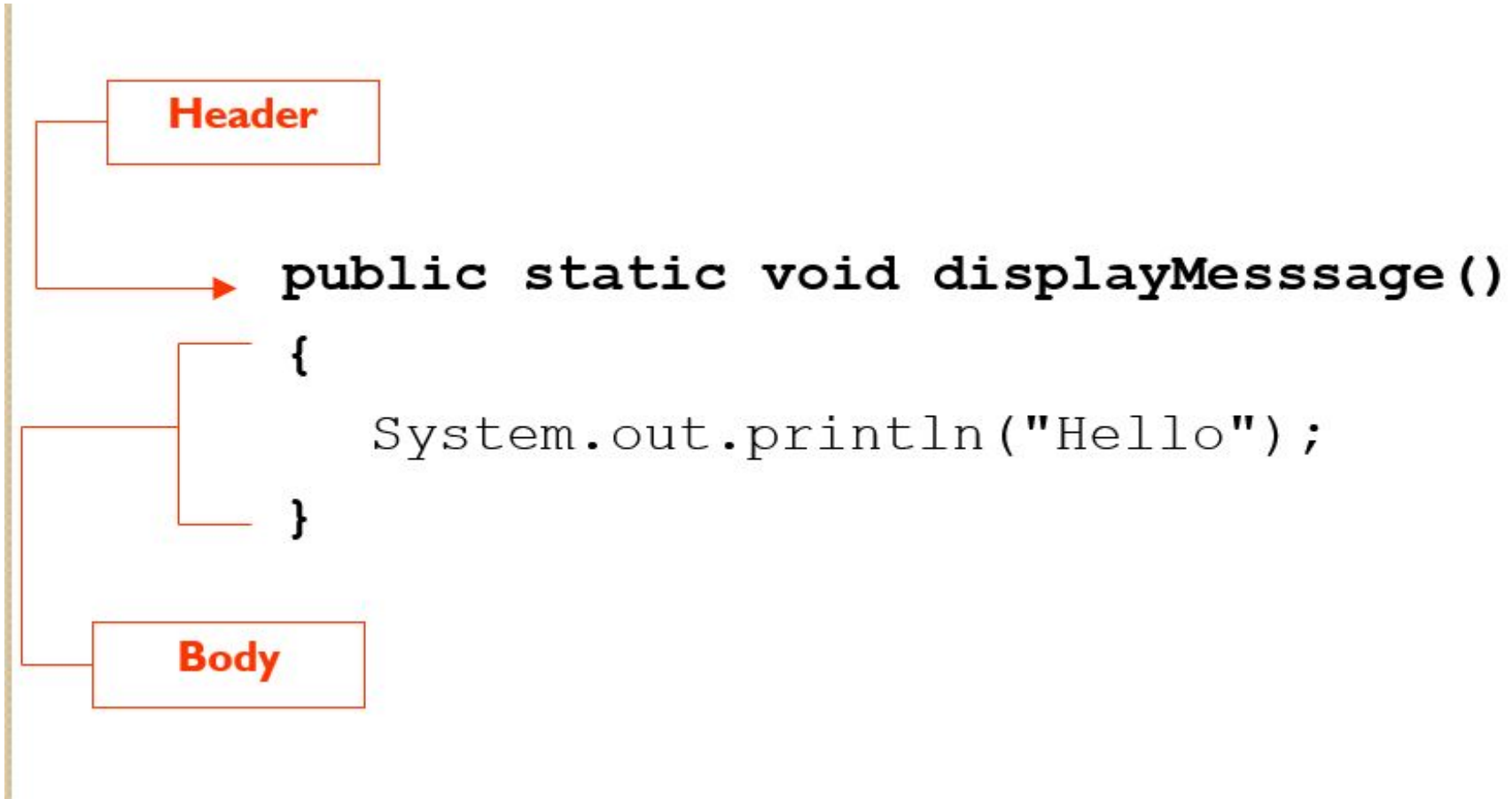
Using the Dog Class and Its Methods

`void` Methods and Value-Returning Methods

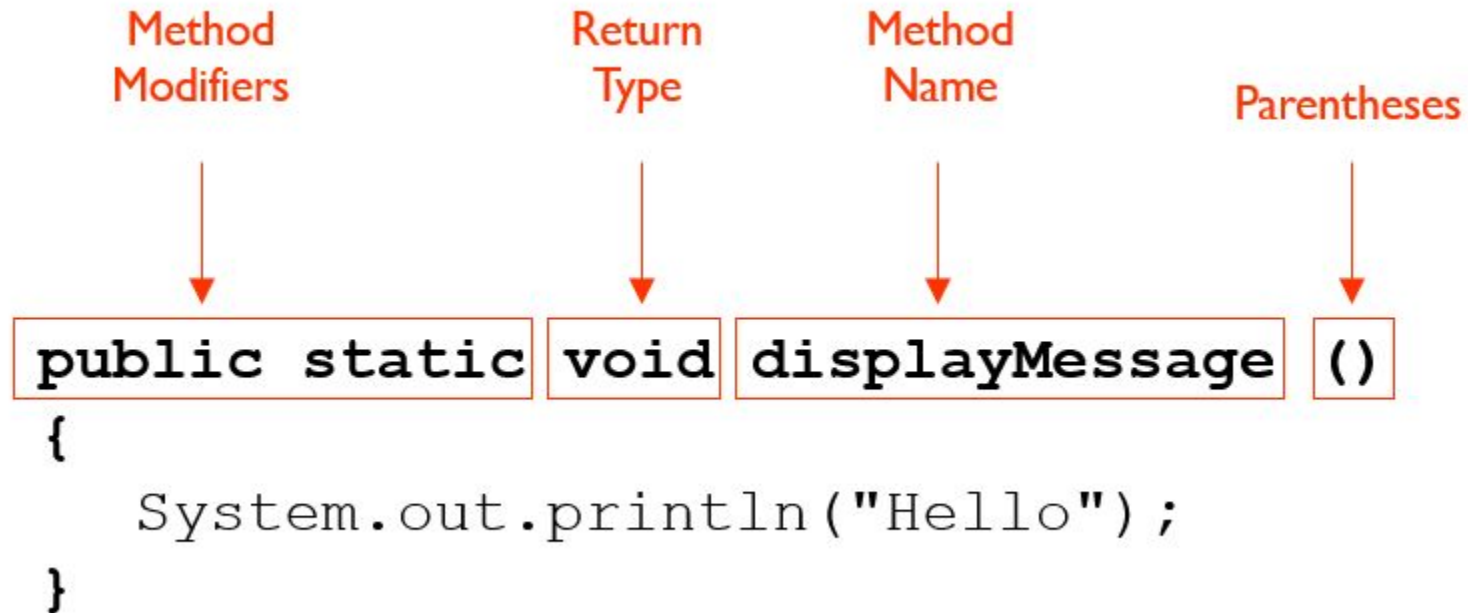
A `void` method is one that simply performs a task and then terminates.

A value-returning method not only performs a task, but also sends a value back to the code that called it.

Two Parts of Method Declaration



Parts of a Method Header



Calling a Method

1. A method executes when it is called.
2. The `main` method is automatically called when a program starts, but other methods are executed by method call statements.

`displayMessage ( ) ;`

Notice that the method modifiers and the `void` return type are not written in the method call statement. Those are only written in the method header.